PAWCYCLE COMMERCE · 리팩터링 준비

기준선 측정과 인수인계

리팩터링 전 성능·기능 기준선을 잰 과정과 결과, 그리고 다른 세션이 이어서 진행하기 위한 정보

기준 커밋	`b3b2235d` (2026-11-13, 백엔드는 f9ab9f87과 동일)
작성일	2026-11-13
범위	상품 목록·상세 처리 한계, 구매 흐름, Gradle 전체 테스트
보존 자산	같은 폴더의 기준선_2026-11-13/ (k6 스크립트, 실행 스크립트, 오버레이, 결과 원본)
실행 범위	원래 저장소는 건드리지 않았다. 작업 폴더 사본과 분리된 Docker 프로젝트에서 로컬로만 실행했다. 비밀 값은 출력하지 않았다

확인 코드·설정·DDL에서 직접 확인 기록 승인 문서·PR·이슈·결정 기록 해석 판단 원리 일반 원리 미확인 확인하지 못함

차례

1. 기준선 측정 결과와 인수인계
   작업 경계, 측정 환경·방법·결과, 측정 중 배운 것, 새 발견, 현재 상태, 다음 할 일

기준선 측정 결과와 인수인계

이 문서의 용도

이 문서는 다른 세션이 이 작업을 **처음부터 다시 조사하지 않고** 이어받기 위한 인수인계 문서다. 2026-11-13에 정한 다음 단계의 작업 경계, 성능·기능 기준선 측정의 과정과 결과, 그리고 다음에 할 일을 담았다.

배경은 같은 폴더의 강의 문서다. 1~34강은 PawCycle Commerce를 해체 분석한 기록이다. 그중 `33강_리팩터링_설계.html`이 리팩터링의 설계와 순서를, `34강_포트폴리오_정리.html`이 포트폴리오 주제를 다룬다. 이 문서는 33강 33-2에서 설계한 "안전망"의 첫 조각인 **성능 기준선**을 실제로 잰 결과다.

---

1. 사용자 결정과 작업 경계

항목	결정(2026-11-13)
다음 단계 1	코드 결함, 도메인 문제, 불필요한 문서, 실무에 맞지 않는 Java/Spring 코드와 디자인 패턴·OOP 위반 코드·구성을 **전수 분석해 정리**한다
다음 단계 2	대규모 데이터 추가 → 성능 측정·부하 테스트 → 성능 개선과 시스템 디자인 재설계를 모두 다시 한다
순서	**기준선 먼저**, 그다음 결함 분석·수정
분석 결과물	분석 폴더 HTML 문서 + GitHub Issue. Issue는 외부 기록이므로 **생성 직전에 목록과 함께 다시 확인**받는다
저장소 쓰기	**아직 분석만.** 파일 수정·커밋·브랜치·PR 없음
로컬 실행	**허용.** Gradle 테스트, Docker Compose, k6
운영·클라우드	손대지 않는다(OCI `app01`, Managed MySQL, GitHub 설정 등)
리팩터링 방향	기존 저장소에서 점진적으로. 이후 기능 도입과 AI 활용으로 확장

지켜야 할 기존 규칙은 다음과 같다.

- 비밀 값(`.env.local`의 비밀번호, 토큰, 웹훅 주소)은 출력하거나 문서에 적지 않는다. 이 문서의 모든 명령도 값을 출력하지 않는다.
- 사용자와 AI의 기여 경계는 추론하지 않는다.
- 근거 수준을 확인 기록 해석 미확인으로 표시한다.

---

2. 측정 대상과 환경

대상 커밋

항목	값
측정 커밋	`b3b2235d` (2026-11-13, 원격 main 최신)
강의 분석 기준	`f9ab9f87` (2026-11-12)
두 커밋의 차이	프론트엔드 UI 커밋 #469 하나뿐. **백엔드 코드는 같다** 확인

원래 로컬 저장소(`C:\Users\guseo\IdeaProjects\pawcycle-commerce`)는 건드리지 않았다. 측정은 GitHub에서 작업 폴더로 새로 받은 사본에서 했다.

> 원래 로컬 저장소에는 이 세션이 만들지 않은 변경이 있다. `docs/reports/OPS-OCI-CD-011/README.md` 수정과 추적되지 않는 `.task-worktrees/`다. 손대지 않았다.

환경 지문

항목	값
호스트	Windows 11 Home
Docker	Docker Desktop, Engine 29.9.2, CPU 8, 메모리 할당 약 3.74GiB
Compose 프로젝트	`pawcycle-baseline` (기존 로컬 볼륨과 분리)
구성	`infra/local-integration/compose.yaml` + `compose.phase9-envelope.yaml` + 카탈로그 오버레이
backend 제한	640MiB, CPU 0.75, PID 256, `MaxRAMPercentage=65`, `ExitOnOutOfMemoryError` (운영과 같은 값)
카탈로그	고객 카탈로그 v3 사용 (`PAWCYCLE_LOCAL_CUSTOMER_CATALOG_V3_ENABLED=true`)
데이터	상품 128개(모두 PUBLIC), SKU 194개, 재고 있는 SKU 179개, 회원 29명
데이터 조작	측정 전용 DB에서 **SKU 7번 재고만 100,000으로** 올렸다
결제	local-integration 기본 **샌드박스 결제 어댑터**(외부 호출 없음, 어떤 결제 키든 성공)
상품 목록 캐시	설정은 켜져 있지만 **코드가 캐시를 읽지 않는다**(28강, PR #279). 이번 숫자는 캐시 없는 상태
부하 도구	k6 v2.2.0

Phase 9(2026-08)의 측정 환경은 Docker 메모리가 약 8GiB였다. 이번에는 약 3.74GiB다. backend 컨테이너 제한은 같지만, MySQL과 다른 컨테이너가 쓸 수 있는 여유가 다르다. 두 측정의 숫자를 직접 비교하지 않는다.

---

3. 측정 방법

시나리오

이름	스크립트	부하 모델	내용
상품 목록	`k6/read.js` `TARGET=list`	예열 2 VU 30초 → `constant-arrival-rate` 60초	`GET /api/products`
상품 상세	`k6/read.js` `TARGET=detail`	같음	`GET /api/products/{id}` 무작위
구매 흐름	`k6/flow.js`	1 VU 순차 60초	로그인 → 담기 → 체크아웃 → 결제 승인 → 주문 목록

`run.sh`는 k6 실행 전후의 MySQL 명령 수(`Com_select`, `Com_insert`, `Com_update`, `Questions`)를 기록한다. 측정 60초 시점에 `docker stats`를 한 번 찍고, 끝난 뒤 backend 스레드 수와 재시작·OOM 여부를 남긴다.

다시 재는 방법

모든 파일은 이 폴더의 `기준선_2026-11-13/`에 있다.

1. GitHub에서 측정할 커밋을 작업 폴더에 받는다. 원래 저장소에서 checkout하지 않는다.
2. 스택을 띄운다(비밀 값은 원래 저장소의 `.env.local`을 경로로만 참조).

```
cd <작업폴더>/repo/infra/local-integration
docker compose -p pawcycle-baseline --env-file <원래저장소>/infra/local-integration/.env.local \
  -f compose.yaml -f compose.phase9-envelope.yaml -f <기준선폴더>/compose.baseline-catalog.yaml \
  up --build -d --wait --wait-timeout 600
```

3. 측정 전용 DB에서 구매 흐름용 재고를 올린다(값은 컨테이너 안의 환경 변수로만 쓴다).

```
docker exec pawcycle-baseline-mysql-1 sh -c 'MYSQL_PWD="$MYSQL_ROOT_PASSWORD" mysql -uroot "$MYSQL_DATABASE" \
  -e "UPDATE inventories SET available_quantity=100000, version=version+1 WHERE sku_id=7;"'
```

4. **JVM을 데운 뒤** 측정한다. 기동 직후 첫 측정은 버린다(5절 참고). 측정 사이에 20~30초 쉰다.

```
bash run.sh list75 read.js -e BASE_URL=http://127.0.0.1:8080 -e TARGET=list
-e RATE=75
bash run.sh detail50w read.js -e BASE_URL=http://127.0.0.1:8080 -e
TARGET=detail -e RATE=50
```

5. 구매 흐름은 QA 계정을 환경 변수로만 넘긴다.

```
( set -a; . <원래저장소>/infra/local-integration/.env.local >/dev/null 2>&1;
set +a
  export QA_EMAIL="$PAWCYCLE_LOCAL_QA_BOOTSTRAP_EMAIL"
QA_PASSWORD="$PAWCYCLE_LOCAL_QA_BOOTSTRAP_PASSWORD"
  bash run.sh flow5 flow.js -e BASE_URL=http://127.0.0.1:8080 -e SKU_ID=7 )
```

---

4. 결과

상품 목록 (데워진 JVM)

목표 RPS	요청 수	dropped	p50	p95	p99	판정	근거
10	601	0	17.8ms	30.5ms	51.8ms	안정	확인
25	1,501	0	16.0ms	37.7ms	75.6ms	안정	확인
35	2,101	0	15.8ms	26.7ms	34.8ms	안정	확인
50	3,001	0	15.5ms	23.5ms	45.4ms	안정	확인
75	4,501	0	14.0ms	28.3ms	66.8ms	안정	확인
100	5,419	581	1,881ms	4,612ms	6,636ms	**붕괴**	확인
150	5,757	3,243	1,997ms	5,363ms	6,378ms	붕괴, 처리 상한 약 95 RPS	확인

100 RPS에서 MySQL CPU가 117%, backend CPU가 한도의 약 100%(0.75 중 0.75)였다. 150 RPS에서는 backend 메모리가 **631MiB/640MiB**까지 올라갔다.

상품 상세 (데워진 JVM)

목표 RPS	요청 수	dropped	p50	p95	p99	판정
25	1,501	0	18.7ms	23.6ms	30.2ms	안정
35	2,100	0	19.5ms	25.8ms	47.5ms	안정
50	3,000	0	18.7ms	25.1ms	52.4ms	안정
75	4,369	132	1,307ms	4,104ms	5,876ms	**붕괴**

요청당 DB 명령 수 (순차 100회, 데워진 상태)

경로	응답 크기	SELECT/요청	전체 명령/요청
상품 목록	12,596 bytes	약 2.1	약 7.4
상품 상세	1,745 bytes	약 9.2	약 14.5

목록의 SELECT 2개는 15강의 "2-query 계약"과 일치한다 확인. SELECT 외의 명령(약 5개)은 트랜잭션 설정·커밋 같은 부가 명령으로 보인다 해석. 어떤 명령인지는 `performance_schema.events_statements_history`로 확인할 후보다 미확인.

구매 흐름 (1 VU, 60초)

항목	값
완주	264회, 오류 0
담기	p50 29ms, p95 89ms
체크아웃	p50 59ms, p95 165ms
결제 승인	p50 76ms, p95 202ms
주문 목록	p50 9ms, p95 52ms
DB 명령/회	INSERT 약 10, UPDATE 약 10, SELECT 약 30

기능 기준선 (Gradle 전체 테스트)

CI와 같은 조건으로 돌렸다. 조건은 MySQL 8.4.10 일회용 컨테이너, CI와 같은 계정·권한, 잠금 관측 옵션(`lock-ci.init.gradle`)이다.

항목	값
테스트 클래스	185개
테스트	4,526개
실패	**4개** (모두 `BillingRecoveryPersistenceParityIntegrationTests`의 같은 파라미터 테스트)
건너뜀	4개
소요 시간	43분 15초(이 PC 기준)

실패 4개의 차이는 `updated_at` 하나다. 기대값은 `23:59:59`, 실제값은 `14:59:59`로 **정확히 9시간** 차이가 난다. 같은 테스트 클래스를 JVM 시간대만 UTC로 바꿔(`utc.init.gradle`) 다시 돌리자 **전부 통과**했다 확인. CI는 UTC로 돌고 이 PC는 한국 시간(UTC+9)이다. 그래서 **테스트 결과가 실행 환경의 시간대에 따라 달라진다.**

시스템 기본 시간대를 쓰는 곳이 운영 코드인지 테스트 픽스처인지는 아직 확인하지 않았다 미확인. 33-3에서 다룬 시간 타입 혼용(`Timestamp`·`LocalDateTime`·`Instant`, `*TimestampJdbcType` 10개)과 같은 뿌리로 보인다 해석.

기준선 판정: 이 PC에서 기능 기준선은 "4,526개 중 4,522개 통과, 실패 4개는 시간대 의존"이다. 리팩터링 전후 비교는 `user.timezone=UTC`로 맞춰서 한다.

---

5. 측정 과정에서 배운 것

기동 직후와 과부하 직후는 따로 봐야 한다

같은 50 RPS가 처음에는 p95 8.7초로 붕괴했고, 데워진 뒤에는 23ms로 안정적이었다. 상품 상세 50 RPS도 같은 일이 있었다.

- **기동 직후**: JIT 컴파일 전이라 요청당 CPU가 크다. 30초·2 VU 예열로는 부족했다.
- **과부하 직후**: Tomcat 스레드가 220개 넘게 늘어난 상태가 한동안 유지된다. 이 상태에서는 평소 버티던 부하에서도 다시 무너진다. 30초쯤 쉬면 스레드가 49개 수준으로 줄었다.

이 현상은 운영에도 그대로 적용된다. **배포 직후**(JVM이 차가움)와 **순간 과부하 직후**가 가장 위험하다 해석. 앞으로 측정할 때는 "첫 측정 버리기"와 "측정 사이 휴식"을 절차로 고정한다.

측정 도구 쪽 실수 세 가지

증상	원인	서버 결함인가
담기가 매번 실패	담기는 성공 시 **204**를 돌려주는데 200·201만 성공으로 봤다	아니다
체크아웃 실패	배송지 응답 필드가 `addressId`인데 `id`로 읽었다	아니다
첫 회차 뒤 401·403	**k6는 기본으로 회차마다 쿠키를 지운다.** `noCookiesReset: true`로 해결	아니다

세 번째는 curl로 같은 흐름을 세 번 연속 돌려 세션이 유지됨을 확인한 뒤에 결론 내렸다 확인. 28강에서 PowerShell 래퍼가 무너졌던 것과 같은 종류의 실패다. 측정 도구를 먼저 검증해야 한다는 교훈이 다시 확인됐다.

Gradle 테스트를 다시 돌리는 방법

```
docker start pawcycle-baseline-testdb      # 처음이면 docker run (포트
127.0.0.1:33306, CI와 같은 계정)
cd <작업폴더>/repo/backend
SPRING_DATASOURCE_URL=jdbc:mysql://127.0.0.1:33306/pawcycle_test \
SPRING_DATASOURCE_USERNAME=pawcycle_ci SPRING_DATASOURCE_PASSWORD=ci-only-
not-a-secret \
./gradlew test --no-daemon --init-script <기준선폴더>/utc.init.gradle
```

테스트 계정 정보는 CI 워크플로에 공개된 일회용 값(`ci-only-not-a-secret`)이다. 운영 비밀이 아니다.

Gradle 테스트 DB 준비

CI는 테스트 계정에 `PROCESS` 권한 말고도 `performance_schema`의 잠금 관측 테이블 조회 권한을 준다. 처음에는 이 단계를 빠뜨려 `AdminCatalogConcurrencyIntegrationTests` 등이 `SELECT command denied ... data_lock_waits`로 실패했다. CI 워크플로의 준비 SQL을 그대로 적용한 뒤 다시 돌렸다. 테스트 결함이 아니다.

---

6. 측정 중 새로 찾은 것

ID	분류	내용	근거
BL-B-01	결함	**주문 목록 API에 페이지가 없다.** `OrderController.list`가 회원의 주문 전체를 `List`로 돌려준다. 측정 회원은 주문 270건이었다	확인
BL-B-02	위험	과부하 때 backend 메모리가 639MiB/640MiB까지 올랐다. 30강 OOM 조건이 그대로다(BL-30-01)	확인
BL-B-03	위험	과부하 뒤 스레드가 줄기 전까지 처리 능력이 떨어진다. Tomcat 최대 스레드 상한이 없다	확인 해석
BL-B-04	개선	요청당 SELECT보다 부가 명령이 많다(목록 2 대 약 5)	확인 미확인 명령 종류
BL-B-05	코드 스타일·구성	기동 로그에 Spring Data Redis가 JPA 리포지토리를 Redis 리포지토리 후보로 검사하는 경고가 리포지토리마다 찍힌다. `spring.data.redis.repositories.enabled=false` 후보	확인
BL-B-06	결함 (재확인)	상품 목록 캐시가 켜져 있어도 읽히지 않는다(BL-28-01). 기준선은 캐시 없는 숫자	확인
BL-B-07	결함	**테스트가 실행 환경의 시간대에 의존한다.** `BillingRecoveryPersistenceParityIntegrationTests`의 4개가 KST에서 9시간 차이로 실패하고 UTC에서 통과한다. 운영 코드인지 테스트 픽스처인지 확인 필요	확인 미확인 원인 위치
BL-B-08	개선	백엔드 전체 테스트가 이 PC에서 43분 걸린다. 리팩터링 중 빠른 피드백을 위해 영역별 실행 묶음(태그)이 필요하다	확인

---

7. 현재 상태

대상	상태
측정 스택 `pawcycle-baseline`	**멈춤**(컨테이너와 볼륨은 남김). 다시 켜려면 3절 2번 명령을 그대로 실행한다. 측정 데이터(SKU 7번 재고, 측정 회원의 주문 약 270건)가 남아 있으므로, 깨끗한 기준선이 필요하면 `docker compose -p pawcycle-baseline down -v`로 볼륨까지 지우고 다시 만든다
테스트 DB `pawcycle-baseline-testdb`	**멈춤.** `docker start pawcycle-baseline-testdb`로 다시 켠다. 권한은 이미 부여돼 있다. 지우려면 `docker rm -f pawcycle-baseline-testdb`
작업 폴더 사본	세션 작업 폴더에 있다. 세션이 끝나면 사라질 수 있다. 다시 받으면 된다
측정 자산	`기준선_2026-11-13/`에 스크립트, 오버레이, 결과 원본(JSON·DB 명령·docker stats), 테스트 요약을 보존했다. 유효한 결과는 `list10·25·35b·50b·75·100·150`, `detail25w·35w·50w·75w`, `flow5`다. `list50`·`detail50`·`detail75`는 기동 직후·과부하 직후 측정(5절), `flow1~4`는 측정 도구 실수 중의 실행이다

---

8. 다음 세션에서 할 일

1. **전수 분석**(단계 1). 범위는 백엔드, 프론트엔드, 인프라, 문서 전체다.
   - 코드 결함과 도메인 문제
   - 불필요하거나 낡은 문서(정본과 실행 보고서의 불일치, 역사 산출물의 검색 소음)
   - 실무에 맞지 않는 Java/Spring 코드, 디자인 패턴·OOP 위반, 구성 문제
   - 출발점: 1~34강 백로그(메모리 `pawcycle-refactoring-backlog`), 33강 33-1 분류표, 이 문서 6절
2. 분석 결과를 33강 형식의 HTML로 만든다.
3. **GitHub Issue 목록을 보여 주고 확인받은 뒤** 생성한다.
4. 저장소 쓰기가 허용되면 33-8 로드맵 순서(안전망 → 공통 기반 → 결제 → 재고 → 정기 → 나머지)로 수정에 들어간다.
5. 단계 2(대규모 데이터)는 이 기준선과 같은 스크립트·절차로 비교한다.

이어서 시작할 때 붙여 넣을 요청 예시

> PawCycle 리팩터링 준비를 이어서 하자. `obsidian-vault\Project\분석\기준선_측정과_인수인계.pdf` (또는 html)를 먼저 읽고, 8절의 1번 전수 분석부터 시작해. 작업 경계는 1절을 따른다.